

LINFO1104
LethOz Company

Année académique :
2023-2024

Auteurs :	**NOMA :**
Lamien Siénou	7264-23-01
Putnik Nikola	6437-17-00

2 mai 2024

1 Introduction

Dans le cadre du cours de concepts, paradigmes et sémantique des langages de programmation (LINFO1104), nous avons contribué au développement de LethOz Company, un jeu stratégique multijoueur où les joueurs contrôlent des vaisseaux spatiaux naviguant dans un champ d'astéroïdes. La contrainte principale dans la réalisation de ce projet en langage Oz était d'utiliser uniquement la programmation déclarative. Ce rapport va parcourir nos démarches dans la réalisation du projet, examiner les problèmes rencontrés, et discuter nos choix d'implémentation ainsi que les extensions que nous avons développées.

2 Conception et Implémentation

2.1 Mouvement des vaisseaux et décodage des stratégies

Nous avons commencé par implémenter les fonctions `fun {Next Spaceship Instruction}` et `fun {DecodeStrategy Strategy}`.

Pour réaliser la fonction `Next`, nous avons implémenté les fonctions `Move` et `Turn`. La fonction `fun {Move Spaceship Direction}` déplace le vaisseau en avançant sa position d'une case dans la direction indiquée en argument. La fonction `{Turn Spaceship RelativeDirection}` permet de gérer les changements de directions. Celle-ci compare la direction actuelle du vaisseau avec la direction relative (left | right) souhaitée dans le but de définir la nouvelle direction du vaisseau sur la grille. Ensuite, elle fait appel à `Move` pour effectuer le déplacement dans la direction trouvée.

La fonction DecodeStrategy permet de définir les mouvements qui doivent être appliqués au vaisseau. La fonction `fun {DecodeInstruction Instruction}` fait appel à la fonction `Next` en donnant la direction se trouvant dans la liste strategy du vaisseau. La fonction `fun {Repeat Strategy Times L}` permet de répéter un mouvement plusieurs fois en appelant la fonction DecodeInstruction. Cette fonction possède 3 arguments : Strategy, Times et L. Strategy est l'instruction qui doit être effectuée : Forward, turn (right) ou turn(left). Times est le nombre de fois que le mouvement doit être effectué. `L` est une liste contenant des fonctions Next.

2.2 Application d'effets dynamiques

Nous avons, implémenté tous les effets requis : scrap, revert, wormhole et dropSeismicCharge.

Une fonction `fun {Bonus Positions Effects B}` a été implémentée pour permettre la gestion des bonus sur la grille de jeu. Avant chaque déplacement, la fonction `Next` fait appel à cette fonction Bonus qui va détecter les effets actifs et appeler les

fonctions correspondantes.

L'effet scrap est effectué en 2 étapes. Premièrement, le vaisseau avance de manière classique de 1. Deuxièmement, la fonction bonus appelle la `fun {BonusACC Positions}` qui permet de renvoyer le dernier élément de la liste des positions du vaisseau avant exécution d'un déplacement. Puis, cet élément est ajouté à la fin de la liste des positions du vaisseau après exécution du mouvement.

L'effet revert utilise la fonction `fun {Reverse L}` qui renverse la liste des positions et met à jour les directions.

L'effet wormhole est la partie qui a posé le plus de difficultés. En particulier dans le fait d'éviter que le vaisseau ne rerentre dans le wormhole après en être sorti. Celui-ci se déroule en plusieurs étapes après avoir détecté que l'effet doit être appliqué. Dans la fonction `Next` une condition permettant de dire si le nombre d'éléments dans la liste des effets est paire ou non. Cette distinction va permettre de savoir si le vaisseau va entrer dans le wormhole ou va sortir. Dans le cas où le vaisseau sortirait du wormhole, le vaisseau avance selon la stratégie. Dans le cas contraire, la fonction bonus est appelé pour changer les coordonnées de la tête du vaisseau en fonction de la position donné dans l'effet wormhole.

L'effet dropSeismicCharge est directement géré dans la fonction `Next` et ajoute true au début de la liste seismicCharge.

3 Extensions

Nous avons implémenté 2 extensions pour le jeu.

3.1 L'effet EMB (ElectroMagnetic bomb)

Cet effet paralyse le vaisseau pour N étapes, ignorant toutes les instructions fournies et pouvant neutraliser d'autres effets actifs.

Techniquement, l'EMB a été implémenté en ajoutant une condition dans la logique de traitement des états du vaisseau qui vérifie la présence de cet effet avant d'exécuter toute autre logique de mouvement ou d'application d'effet. Si l'EMB est actif, le système ignore toutes les instructions entrantes et maintient l'état du vaisseau sans changement, tout en comptant le nombre d'étapes jusqu'à l'expiration de l'effet.

3.2 L'effet Ghost

L'effet Ghost permet de traverser les wormholes sans être téléporté. Niveau implémentation, une condition a été rajouté dans le bonus wormhole permettant

de voir si la liste des effets contient Ghost. Si oui, le vaisseau avance normalement comme convenu dans la stratégie du vaisseau. Sinon, le vaisseau est téléporté ce qui représente l'exécution classique du wormhole . Voici ci-dessous un exemple de l'effet Ghost et de l'effet EMB. Dans l'image 1, l'extension Ghost est en couleur bordeaux, en mauve, c'est un wormhole, en vert l'extension EMB. L'image 2 montre le vaisseau qui prend l'extension Ghost. L'image 3 montre le vaisseau avançant sur le wormhole. L'image 4 montre le vaisseau qui n'est pas téléporté par le wormhole, mais avance normalement de 1 vers le nord. Cette image montre l'application de l'effet Ghost sur le vaisseau. Sur les images 5 et 6, on voit que le vaisseau n'a pas bougé. Ceci est dû à l'application de l'effet EMB qui stoppe le vaisseau pendant 2 steps. L'image 7 montre qu'une fois l'effet EMB fini le vaisseau avance conforment à la stratégie.

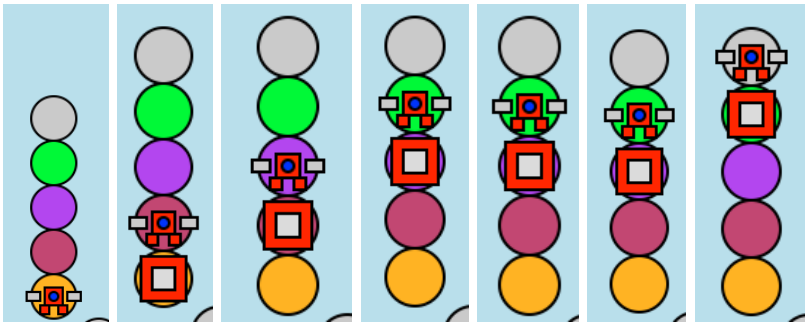


FIGURE 1 – L'effet Ghost et EMB

4 Complexité

La majorité des fonctions que nous avons implémenté opèrent avec une complexité linéaire ou constante.

Par exemple, les fonctions `Move` et `Turn` ne font que quelques opérations basées sur des cas fixes, elles ont donc des complexités temporelles constantes O(1).

La fonction `Next` a une complexité linéaire. Sa complexité temporelle est principalement O(n), où n est le nombre de segments. Sa complexité spatiale est également O(n), principalement due à la création de nouvelles structures pour représenter l'état mis à jour du vaisseau.

5 Scénario

Nous avons développé un scénario de test complet pour démontrer toutes les fonctionnalités et extensions implémentées.

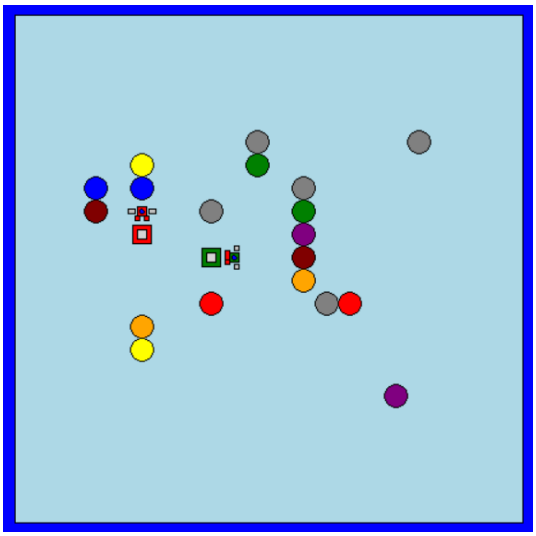


FIGURE 2 – EMB en vert ; Ghost en marron ; Wormhole en jaune, violet et orange ; Revert en rouge ; Scrap en gris ; Charge sismique en bleu.

6 Conclusion

Pour conclure, nous avons réussi à implémenter toutes les fonctionnalités de base ainsi que deux extensions pour étendre davantage les possibilités du jeu LethOz Company. Ce projet a été une expérience enrichissante, qui nous a permis d'améliorer nos capacités de programmation en Oz et de mettre en application les principes de programmation déclarative dans un contexte pratique.